Third Programming Assignment

MSDS 458

February 20, 2026

Lindsey Chenault

Abstract

The AG News dataset was used to examine how neural network architecture influences performance in multiclass text classification. The dataset contains short news summaries categorized into four classes: World, Sports, Business, and Science and Technology. Its balanced class distribution and relatively short text samples make it especially useful for comparing models in a controlled and systematic way. Differences can be examined without excessive noise from uneven class representation or long, highly variable documents because the dataset is so structured and consistent.

To support a fair and momentous comparison, fifteen neural network models were trained using the same experimental setup, which makes certain that each architecture was evaluated under identical conditions. Core hyperparameters, including sequence length, embedding dimension, batch size, optimizer selection, and loss function, were held constant across all experiments to ensure a stable and consistent training environment. By structuring the experiment in this way, architectural design was treated as the central variable of interest. This deliberate focus enhances the credibility and interpretability of the findings, as differences in model performance can be attributed more directly to structural characteristics rather than variations in training procedures or optimization strategies.

The models included a baseline dense network, several SimpleRNN variants, a bidirectional RNN, multiple Long Short Term Memory configurations, including stacked and bidirectional forms, and one-dimensional convolutional neural networks. Performance was considered using validation accuracy to compare how effectively each architecture learned and generalized in a multiclass classification setting.

Beyond architectural comparisons, preprocessing experiments assessed the effects of vocabulary size (1,000, 5,000, and 20,000 tokens), edited versus unedited vocabularies, and default versus fixed-length sequence encodings. Model performance was evaluated using training, validation, and test accuracy and loss, as well as total training time to quantify computational efficiency. The dense baseline achieved the highest test accuracy (0.895) while needing less training time than architectures designed to encode word order. Although recurrent and convolutional models demonstrated stable learning dynamics, increased complexity did not result in proportional improvements in generalization. These discoveries mean that when categories are defined by specific keywords, simpler neural networks deliver an optimal balance of accuracy and efficiency.

Introduction

The exponential growth of digital text has heightened the need for automated systems capable of efficiently organizing and classifying large-scale document collections. News topic classification is a foundational task in natural language processing (NLP), with applications in recommendation systems, automated content moderation, filtering, and customer support triage. Increasingly sophisticated architectures, including recurrent, convolutional, and attention-based models, have become prevalent as deep learning has emerged as the dominant approach in NLP. However, as noted in practical deep learning literature, increased architectural complexity does not inherently yield better generalization; performance improvements depend on the alignment between model structure and task-specific characteristics (Chollet, 2018).

This study addresses the central empirical question: Does increased architectural complexity enhance generalization performance in short-form news classification when hyperparameters are held constant? Recurrent and gated architectures, such as RNNs and

LSTMs, are designed to model sequential structure and long-range dependencies. Convolutional neural networks (CNNs) can extract localized n-gram and phrase-level features through learned filters. While these mechanisms are theoretically beneficial for tasks requiring extended contextual modeling, certain classification problems, particularly those involving short, information-dense documents, may rely mostly on salient lexical indicators instead of complex sequential dependencies. In such cases, additional memory mechanisms or multi-level feature extraction may increase computational overhead without corresponding gains in predictive performance.

A dense neural network serves as a robust baseline because it does not model word order; instead, it learns patterns from aggregated word embeddings to herald class labels. In contrast, sequential models are designed to capture word order and contextual structure using recurrent neural networks (RNNs and LSTMs) or local feature extraction (CNNs). Evaluating these models under identical training conditions provides a controlled framework to assess whether increased complexity yields significant improvements in test performance. Both predictive accuracy and training time are analyzed to evaluate the trade-off between model complexity and practical efficiency.

Literature Review

Recent deep learning research addresses the challenge of selecting or integrating architectures to capture both local discriminative patterns, like short phrases, key n-grams, or salient features, and long-range or sequential dependencies, including contextual connections across sentences or temporal memory in time-series data. Zhang and Abdullah et al. approach this design problem from two complementary perspectives: hybrid modeling and comparative

benchmarking. Together, these studies establish a conceptual framework for evaluating deep architectures based on classification performance.

Zhang's research on text classification argues that reliance on a single architecture may limit overall performance. LSTM networks are well-suited for capturing long-term dependencies in text sequences, while CNNs effectively extract local features predictive of class labels, such as short phrases or keyword patterns. In order to manage this limitation, Zhang introduces a combined LSTM–CNN model that preserves global context through the recurrent component, identifies local cues via convolution, and integrates these representations for classification. Experimental results reveal that this composite method outperforms simpler baselines, including standalone CNN and LSTM models as well as traditional feature-based methods, on widely used public text benchmarks. These findings support the claim that jointly modeling sequential context and local patterns produces more robust text representations for classification (Zhang, 2025).

On the other hand, Abdullah et al. investigate which deep learning models are most appropriate for specific tasks by evaluating CNN, LSTM, and GRU architectures across multiple datasets involving both classification and prediction problems. Their findings reveal a practical alignment between task type and architectural design: CNNs generally achieve the highest classification accuracies, while LSTMs excel in prediction tasks involving sequential or time-dependent data. GRUs are identified as an effective compromise that performs competitively across both categories. This suggests that the GRU's gating mechanisms and streamlined memory structure provide a favorable balance between classification performance and sequence modeling capacity (Abdullah et al., 2024).

Both studies analyze how architectural choices, which include CNN, recurrent, or hybrid models, affect performance in controlled comparisons. Zhang proposes a model-integration strategy by hybridizing to capture complementary signal types in text, while Abdullah et al. provide a benchmarking rationale for evaluating such hybrids, which emphasizes that model strengths depend on the specific task and data structure, such as static pattern extraction versus sequential dependence (Abdullah et al., 2024; Zhang, 2025).

Methods

The AG News dataset from TensorFlow Datasets was employed, which has four equally represented topic categories: *World, Sports, Business,* and *Science/Technology*. Each category includes 31,900 documents, which resulted in a balanced four-class classification task. Exploratory data analysis indicated that the corpus consists predominantly of short news summaries rather than full-length articles. The median tokenized length is approximately 20 tokens, with a maximum of 95 tokens. Most samples fall between 15 and 25 tokens, which reflects low variance in document length and limits the influence of extreme-length outliers on model training. This distribution necessitates correct classification based on relatively limited contextual information. The relatively narrow distribution of sequence lengths reduces variability introduced by padding and minimizes the likelihood that performance differences are driven by document-length artifacts. This characteristic strengthens the validity of architectural comparisons.

Input sequences were standardized by padding and truncating to a fixed length of 128 tokens. As the longest sequence observed was 95 tokens, truncation was minimal, and padding primarily ensured uniform tensor dimensions. This standardising enabled direct architectural comparison without substantial information loss. Because the fixed sequence length exceeded the

maximum observed document length, the padding strategy introduced structural uniformity without truncating meaningful content. As a result, representational ability rather than preprocessing constraints determined model behavior. Vocabulary size was varied at 1,000, 5,000, and 20,000 tokens to evaluate the impact of suggestive ability on classification performance. This variation allowed examination of the trade-off between lexical coverage and sparsity, as larger vocabularies increase symbolic richness but may introduce infrequent tokens with limited predictive contribution. Additional preprocessing compared vocabularies with common stopwords removed to those retaining the most frequent tokens. The effect of TensorFlow's default output sequence length was assessed relative to the manually fixed 128-token setting to determine whether sequence standardization influenced learning stability or accuracy.

Fifteen experimental models were assessed, comprising eleven architectural and four preprocessing variations. All models were implemented in TensorFlow and Keras, with GPU acceleration to ensure consistent computational conditions. To isolate architectural effects, core hyperparameters were held constant: sequence length of 128 tokens, embedding dimension of 256, batch size of 100, RMSprop optimizer, sparse categorical cross-entropy loss, and a maximum of 10 training epochs with early stopping based on validation performance. The TextVectorization layer was adapted solely on the training data to prevent data leakage. Training duration for each configuration was recorded using Python timing utilities to enable analysis of computational effectiveness alongside predictive performance.

The evaluated architectures included a dense feedforward baseline, several SimpleRNN variants with differing hidden-unit sizes and regularization techniques, a bidirectional SimpleRNN, unidirectional and bidirectional LSTM models, a stacked LSTM configuration to

examine hierarchical sequence modeling, and one-dimensional CNN models with varying convolutional depths and filter counts. This progression enabled a structured comparison among non-sequential, shallow-sequential, deep-sequential, and convolutional feature-extraction methods. Ordering models from least to most sequence-aware provided an organized framework for assessing whether increased contextual representation capacity produces measurable gains in generalization performance. For all models, training and validation accuracy and loss curves were generated to evaluate convergence dynamics, optimization stability, and evidence of overfitting or underfitting across experimental conditions. These diagnostic visualizations supported the interpretation of both quantitative results and qualitative learning patterns, including the rate of convergence and divergence between training and validation trajectories.

Results

The dense baseline achieved the highest test accuracy of 0.895 and a test loss of 0.309, and it only needed 354 seconds of training time. These results identify it as both the most accurate and computationally effective model evaluated. Training accuracy increased rapidly during the initial epochs, which shows effective feature learning from lexical inputs. Validation accuracy peaked early and then gradually declined, a pattern characteristic of overfitting; however, early stopping effectively limited performance degradation. Although the dense architecture lacks explicit temporal modeling mechanisms, it demonstrated strong generalization performance, which suggests that class distinctions are largely encoded in lexical feature distributions instead of extended sequential dependencies. This result alludes to the fact that the dataset's predictive signal is primarily seized through aggregated lexical features rather than sequential dependencies.

SimpleRNN models achieved test accuracies between 0.880 and 0.891, but training times often exceeded 1,000 seconds, which results in a substantial increase in computational cost compared to the dense baseline. Increasing the number of recurrent units improved training accuracy and reduced training loss, demonstrating greater representational ability. However, these improvements did not consistently translate to the test set, which suggests that additional recurrent capacity primarily improved the model fitting rather than generalization. Bidirectional RNN variants produced modest improvements in validation stability compared to unidirectional models, which shows their ability to incorporate information from both forward and backward token sequences. Nevertheless, these improvements were incremental and did not yield superior test accuracy relative to the dense model.

LSTM-based architectures yielded test accuracies ranging from 0.860 to 0.865. Compared to standard RNNs, LSTMs exhibited smoother convergence and narrower gaps between training and validation accuracy, consistent with their gated memory structure mitigating gradient instability. However, these stability advantages came at a considerable computational cost. The stacked LSTM configuration required over 4,300 seconds of training yet failed to surpass the dense model's performance. The absence of measurable gains hints that the long-term memory mechanisms designed to capture extended sequential patterns were underutilized in this dataset, likely because AG News documents are compact and information-dense rather than dependent on extended sequential structure. With training times lower than those of LSTMs but higher than those of the dense baseline, CNN architectures achieved test accuracies between 0.850 and 0.852. Convolutional filters efficiently captured local n-gram patterns, and convergence was generally stable. However, the performance ceiling

remained below that of the dense architecture, which means that explicit local pattern extraction did not provide an advantage beyond what dense embeddings already captured.

The relatively narrow distribution of document lengths may have limited the benefit of layered feature extraction, which is usually associated with deeper CNNs. Preprocessing experiments further supported the conclusion that increased representational complexity produced diminishing returns. Expanding the vocabulary beyond 5,000 tokens yielded minimal improvements in validation and no consistent increase in test accuracy, which insinuates that most of the discriminative signal resides within a relatively compact lexical subset. Removing common stopwords produced negligible performance differences, which implies that stopwords either contribute little noise in this dataset or that the models inherently learn to downweight them. Comparisons between default sequence encoding and 128-token fixed-length padding produced nearly identical results, which ultimately ratifies that truncation and padding choices had minimal impact given the short document lengths. Collectively, these results indicate that AG News classification performance is primarily determined by salient lexical features and short contextual spans, rather than long-term sequential dynamics. This finding accounts for the superior generalization efficiency observed in simpler architectures.

Conclusions

The results of this study point to the fact that increasing the complexity of neural network architectures does not necessarily improve performance on short-form news classification tasks. Across fifteen controlled trials, the dense baseline model attained the highest test accuracy and required the least computational time. In contrast, sequence-aware architectures like SimpleRNNs and LSTMs substantially increased training duration and parameter complexity, which yields only modest improvements in test-set generalization. Similarly, CNN variants,

despite their layered feature extraction abilities, did not outperform the dense model. These results show that architectural depth and gating mechanisms do not guarantee better predictive outcomes when the task structure does not necessitate extended contextual representation. The observed differences reflect systematic architectural effects rather than random variation, because the performance gap between the dense baseline and sequence-based models remained consistent across the validation and test sets. This stability confirms that architectural complexity alone does not guarantee better performance in short-text classification settings.

These findings underscore a pivotal modeling principle: aligning model design with data characteristics is more effective than pursuing architectural elaborateness. In the AG News dataset, documents are concise, and class boundaries are primarily defined by discriminative lexical cues rather than long-range semantic dependencies. Feedforward architectures operating on embedded representations capture sufficient information for high-performance classification under these circumstances. Incorporating additional recurrent memory mechanisms or convolutional hierarchies increases representational capacity that remains underutilized, which results in higher computational costs without proportional gains in predictive accuracy. These results highlight how important it is to assess whether a dataset contains meaningful distant dependencies before implementing memory-based architectures. For compact, information-dense documents, additional sequential modeling capacity may introduce redundancy rather than predictive benefit.

From a systems and management perspective, these results have practical implications for the design of conversational agents, enterprise natural language processing pipelines, and real-time classification services. Tasks like intent detection, document routing, content tagging, and short-message triage often involve brief inputs. In these contexts, selecting simpler

architectures can reduce infrastructure requirements, training time, inference latency, and power consumption while maintaining comparable accuracy. Reduced model complexity also facilitates deployment, monitoring, and retraining, thereby improving maintainability and minimizing technical debt.

Strategic model selection constitutes both a resource-allocation and a predictive decision. Complex architectures should be adopted only when measurable performance improvements justify their costs for processing and operations. When the gains in margin and precision are minimal, the trade-offs may not be warranted in relation to scalability, hardware utilization, and maintenance. Effective system design needs to balance predictive performance with efficiency, scalability, and long-term viability. When choosing the right models, they need to line up with both theoretical and methodological capabilities and the practical constraints of production environments. Ultimately, validation needs to guide architectural decisions rather than theoretical appeal, particularly in applied contexts where performance gains must justify the cost of resources. Controlled experimentation, as demonstrated in this study, extends an applicable framework for informing such decisions.

References

Abdullah, H. S., Ali, N. H., & Abdullah, N. A. Z. (2024). Evaluating the performance and behavior of CNN, LSTM, and GRU for classification and prediction tasks. *Iraqi Journal of Science, 65*(3), 1741–1751. https://ijs.uobaghdad.edu.iq/index.php/eijs/article/view/9952

Chollet, F. (2018). *Deep learning with Python*. Manning Publications. https://www.manning.com/books/deep-learning-with-python

Zhang, H. (2025). Research on text classification based on LSTM-CNN. In *Proceedings of the 2024 5th International Conference on Computer Science and Management Technology (ICCSMT '24)* (pp. 277–282). Association for Computing Machinery. https://dl.acm.org/doi/full/10.1145/3708036.3708084

Appendix

Results Table

	Experiment	Family	Params(M)	TrAcc	VaAcc	TeAcc	TrLoss	VaLoss	TeLoss	Time(s)
0	A0_Dense_Flatten	Dense	3.377	0.983	0.851	0.895	0.059	0.846	0.309	354
1	B1_SimpleRNN_32	RNN	1.289	0.914	0.886	0.886	0.290	0.373	0.354	1075
2	B2_BiSimpleRNN_32	RNN	1.299	0.941	0.889	0.891	0.189	0.395	0.336	2229
3	B3_SimpleRNN_64	RNN	1.301	0.924	0.884	0.880	0.245	0.391	0.363	1226
4	B4_SimpleRNN_L2	RNN	0.265	0.853	0.841	0.846	0.454	0.469	0.457	787
5	C1_LSTM_32	LSTM	0.293	0.875	0.865	0.860	0.359	0.397	0.397	1807
6	C2_BiLSTM_32	LSTM	0.330	0.885	0.867	0.863	0.333	0.392	0.395	3507
7	C3_BiLSTM_32_Dense64	LSTM	0.334	0.879	0.865	0.865	0.348	0.405	0.396	3215
8	C4_Stacked_LSTM	LSTM	0.351	0.887	0.870	0.861	0.323	0.402	0.403	4388
9	D1_CNN_1Layer	CNN1D	0.290	0.849	0.853	0.852	0.428	0.449	0.452	386
10	D2_CNN_Deeper	CNN1D	0.335	0.861	0.860	0.850	0.396	0.412	0.419	616

Vocabulary Size: 95827

['' '[UNK]' '39s' 'said' 'new' 'us' 'reuters' 'ap' 'two' 'first' 'monday'

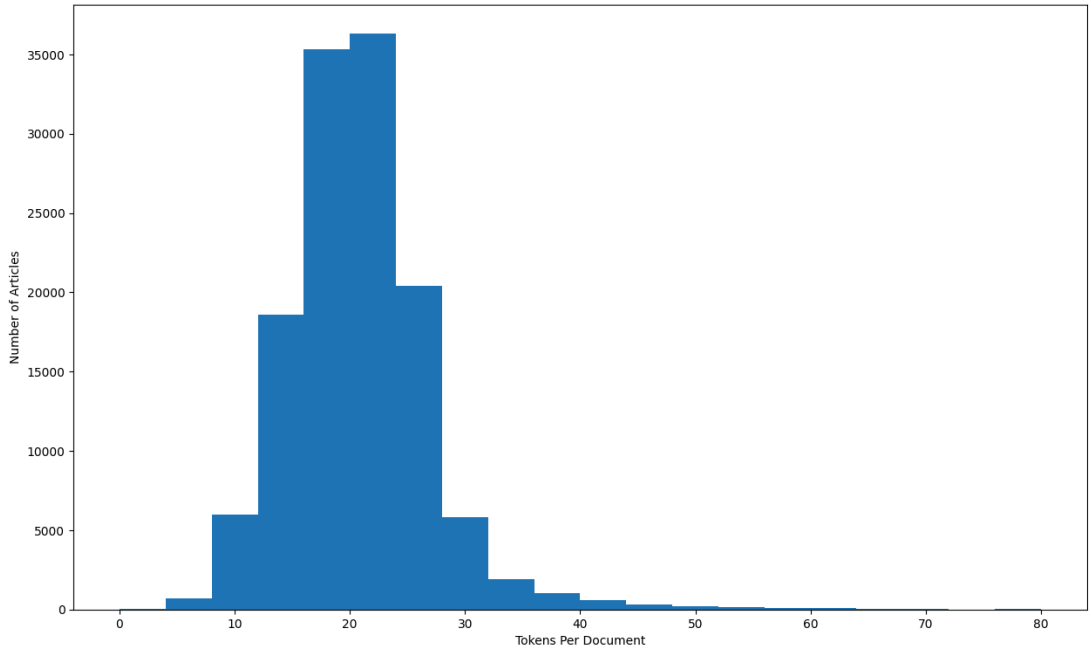
'wednesday' 'tuesday' 'thursday' 'company' 'friday' 'inc' 'one' 'world'

'yesterday' 'last' 'york' 'year' 'president' 'million' 'oil' 'corp'

'united' 'would' 'sunday' 'years' 'week' 'people' 'today' 'three'

'government' 'could' 'quot' 'group' 'time' 'percent' 'game' 'saturday'

'software' 'night' 'next' 'prices' 'iraq' 'security' 'announced']



Experiment A(a):

Vocabulary Size = 1000

Actual Vocab Size: 1000

Top 20 Tokens: ['', '[UNK]', np.str_('39s'), np.str_('said'), np.str_('new'), np.str_('us'), np.str_('reuters'), np.str_('ap'), np.str_('two'), np.str_('first'), np.str_('monday'), np.str_('wednesday'), np.str_('tuesday'), np.str_('thursday'), np.str_('company'), np.str_('friday'), np.str_('inc'), np.str_('one'), np.str_('world'), np.str_('yesterday')]

Vocabulary Size = 5000

Actual Vocab Size: 5000

Top 20 Tokens: ['', '[UNK]', np.str_('39s'), np.str_('said'), np.str_('new'), np.str_('us'), np.str_('reuters'), np.str_('ap'), np.str_('two'), np.str_('first'), np.str_('monday'),

np.str_('wednesday'), np.str_('tuesday'), np.str_('thursday'), np.str_('company'), np.str_('friday'), np.str_('inc'), np.str_('one'), np.str_('world'), np.str_('yesterday')]

Vocabulary Size = 20000

Actual Vocab Size: 20000

Top 20 Tokens: ['', '[UNK]', np.str_('39s'), np.str_('said'), np.str_('new'), np.str_('us'), np.str_('reuters'), np.str_('ap'), np.str_('two'), np.str_('first'), np.str_('monday'), np.str_('wednesday'), np.str_('tuesday'), np.str_('thursday'), np.str_('company'), np.str_('friday'), np.str_('inc'), np.str_('one'), np.str_('world'), np.str_('yesterday')]

Experiment A(b):

UNEDITED VOCAB

Top 20: ['', '[UNK]', np.str_('the'), np.str_('to'), np.str_('a'), np.str_('of'), np.str_('in'), np.str_('and'), np.str_('on'), np.str_('-'), np.str_('for'), np.str_('that'), np.str_('#39;s'), np.str_('The'), np.str_('with'), np.str_('its'), np.str_('as'), np.str_('is'), np.str_('at'), np.str_('has')]

EDITED VOCAB

Top 20: ['', '[UNK]', np.str_('39s'), np.str_('said'), np.str_('new'), np.str_('us'), np.str_('reuters'), np.str_('ap'), np.str_('two'), np.str_('first'), np.str_('monday'), np.str_('wednesday'), np.str_('tuesday'), np.str_('thursday'), np.str_('company'), np.str_('friday'), np.str_('inc'), np.str_('one'), np.str_('world'), np.str_('yesterday')]

Experiment A(c):

DEFAULT SEQUENCE LENGTH

Encoded Length (Default): 17

FIXED SEQUENCE LENGTH = 128

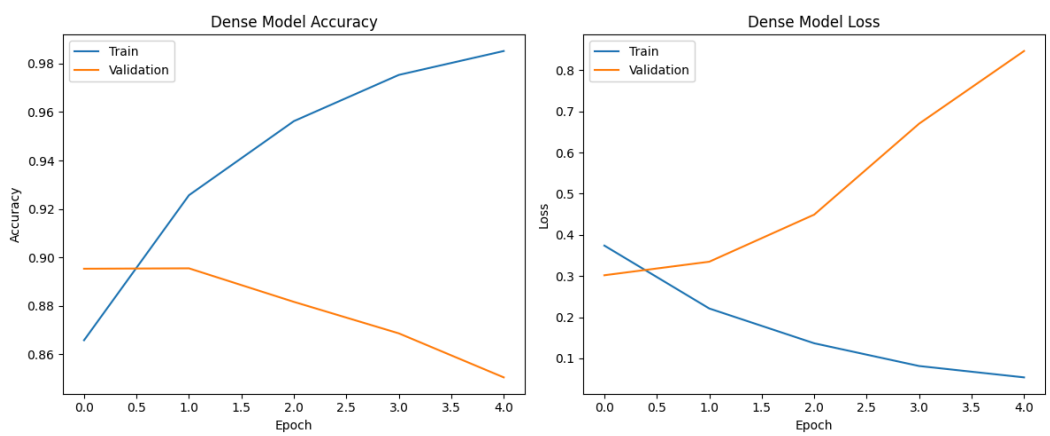
Encoded Length (Fixed 128): 128

Dense Fully Connected Baseline Model:

Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, 128)	0
embedding (Embedding)	(None, 128, 256)	1,280,000
flatten (Flatten)	(None, 32768)	0
dense (Dense)	(None, 64)	2,097,216
dense_1 (Dense)	(None, 4)	260

Total params: 3,377,476 (12.88 MB)
Trainable params: 3,377,476 (12.88 MB)
Non-trainable params: 0 (0.00 B)

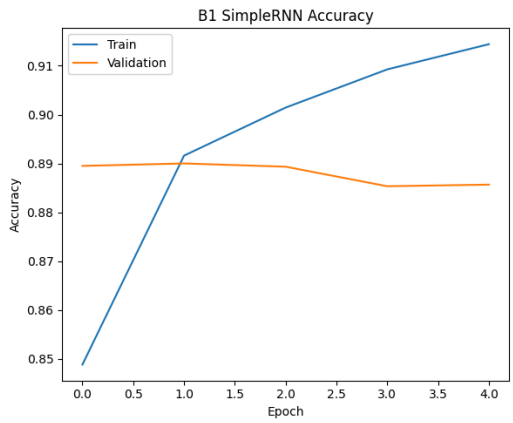


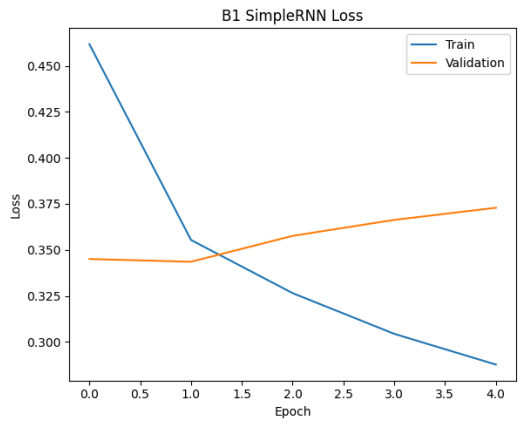
Experiment B (1):

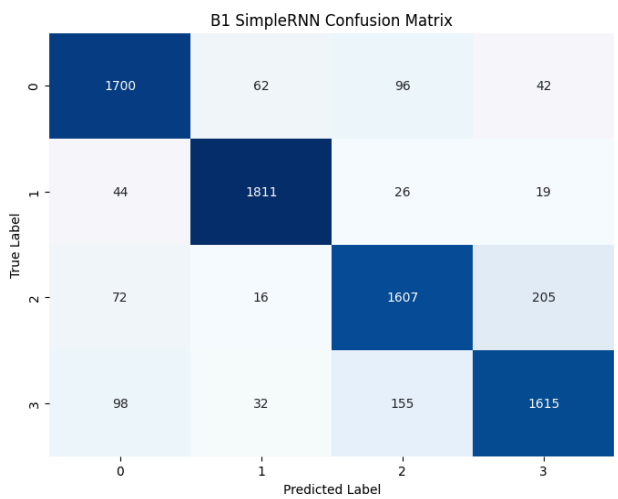
Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, None)	0	–
embedding (Embedding)	(None, None, 256)	1,280,000	input_layer[0][0]
not_equal (NotEqual)	(None, None)	0	input_layer[0][0]
simple_rnn (SimpleRNN)	(None, 32)	9,248	embedding[0][0], not_equal[0][0]
dropout (Dropout)	(None, 32)	0	simple_rnn[0][0]
dense (Dense)	(None, 4)	132	dropout[0][0]

Total params: 1,289,380 (4.92 MB)
Trainable params: 1,289,380 (4.92 MB)
Non-trainable params: 0 (0.00 B)





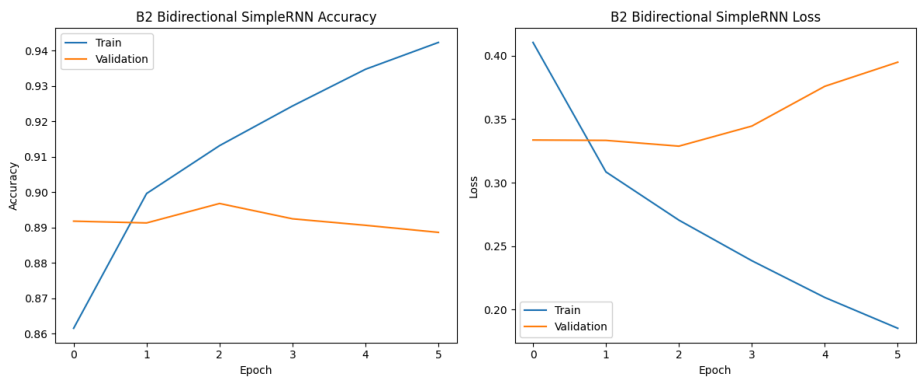


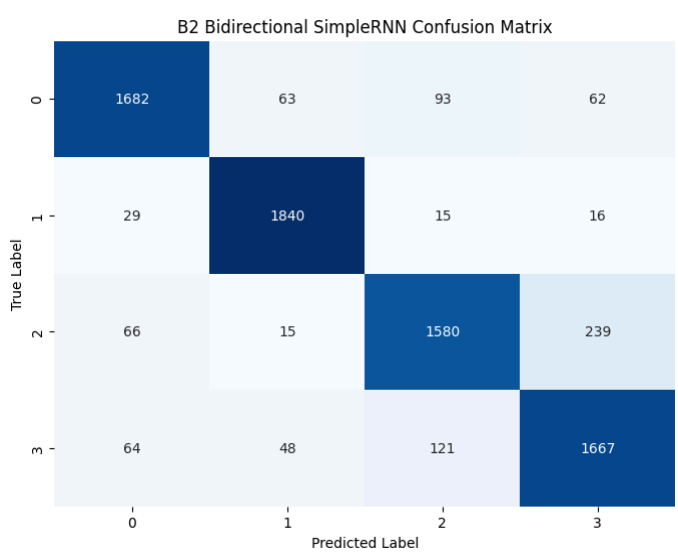
Experiment B(2):

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, None)	0	–
embedding (Embedding)	(None, None, 256)	1,280,000	input_layer[0][0]
not_equal (NotEqual)	(None, None)	0	input_layer[0][0]
bidirectional (Bidirectional)	(None, 64)	18,496	embedding[0][0], not_equal[0][0]
dropout (Dropout)	(None, 64)	0	bidirectional[0]…
dense (Dense)	(None, 4)	260	dropout[0][0]

Total params: 1,298,756 (4.95 MB)
Trainable params: 1,298,756 (4.95 MB)
Non-trainable params: 0 (0.00 B)



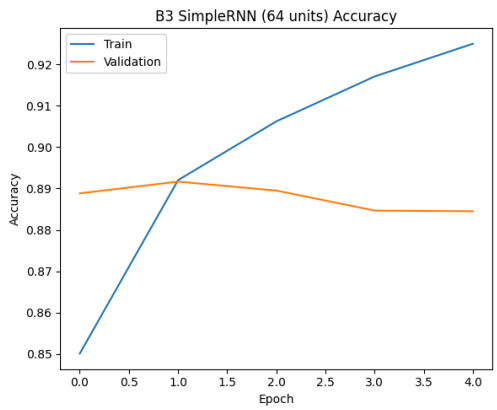


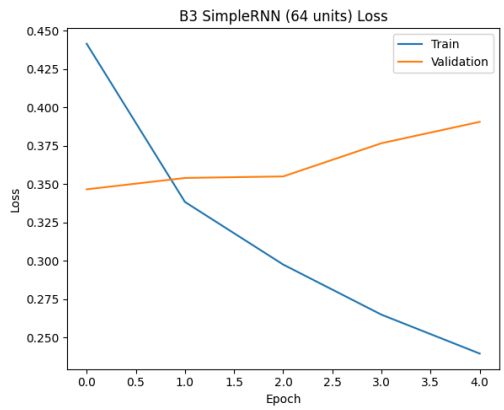
Experiment B(3):

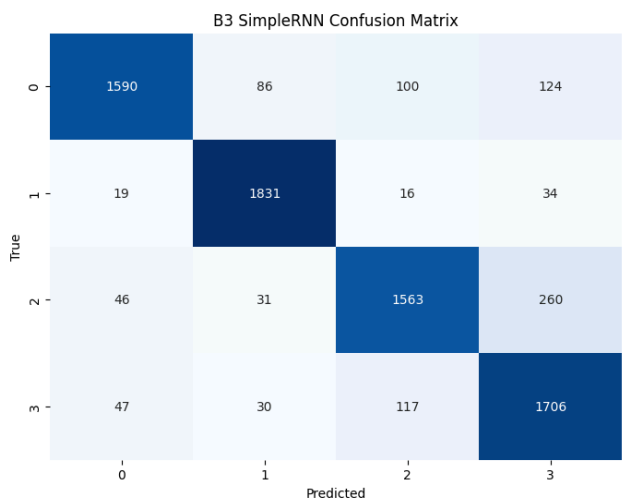
Model: "B3_SimpleRNN_64u"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 128)	0	–
embedding (Embedding)	(None, 128, 256)	1,280,000	input_layer[0][0]
not_equal (NotEqual)	(None, 128)	0	input_layer[0][0]
simple_rnn (SimpleRNN)	(None, 64)	20,544	embedding[0][0], not_equal[0][0]
dropout (Dropout)	(None, 64)	0	simple_rnn[0][0]
dense (Dense)	(None, 4)	260	dropout[0][0]

Total params: 1,300,804 (4.96 MB)
Trainable params: 1,300,804 (4.96 MB)
Non-trainable params: 0 (0.00 B)





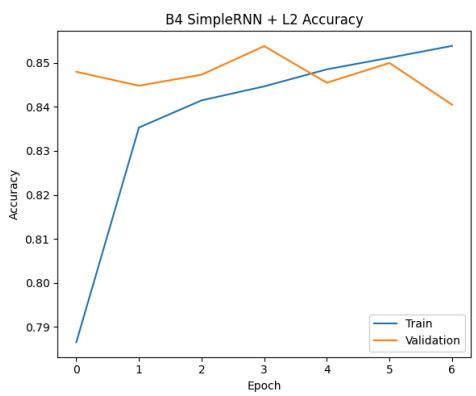


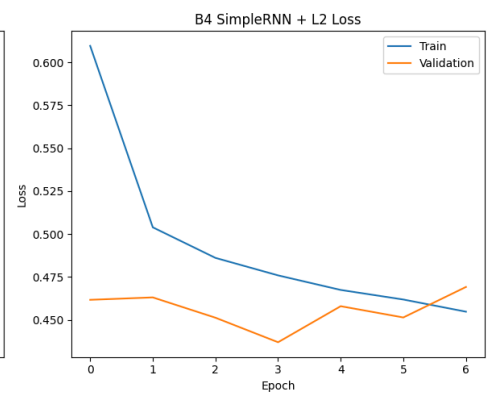
Experiment B(4):

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, None)	0	-
embedding (Embedding)	(None, None, 256)	256,000	input_layer[0][0]
not_equal (NotEqual)	(None, None)	0	input_layer[0][0]
simple_rnn (SimpleRNN)	(None, 32)	9,248	embedding[0][0], not_equal[0][0]
dropout (Dropout)	(None, 32)	0	simple_rnn[0][0]
dense (Dense)	(None, 4)	132	dropout[0][0]

Total params: 265,380 (1.01 MB)
Trainable params: 265,380 (1.01 MB)
Non-trainable params: 0 (0.00 B)



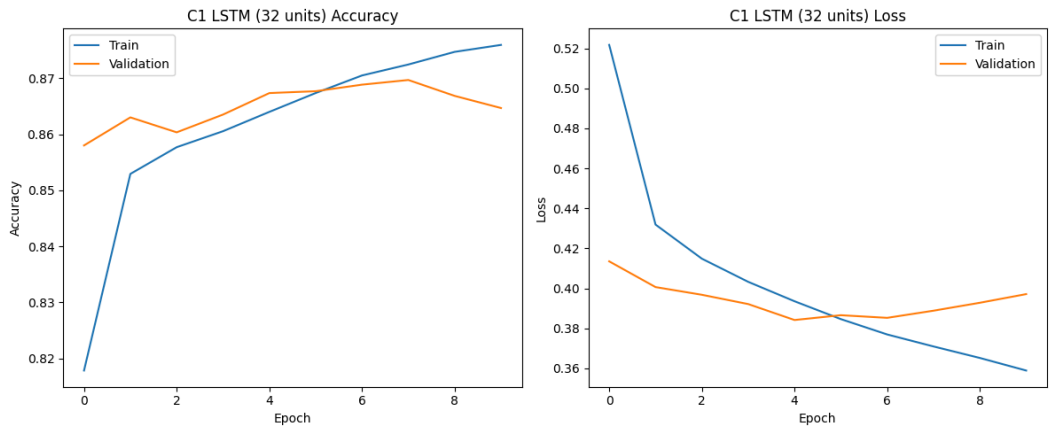


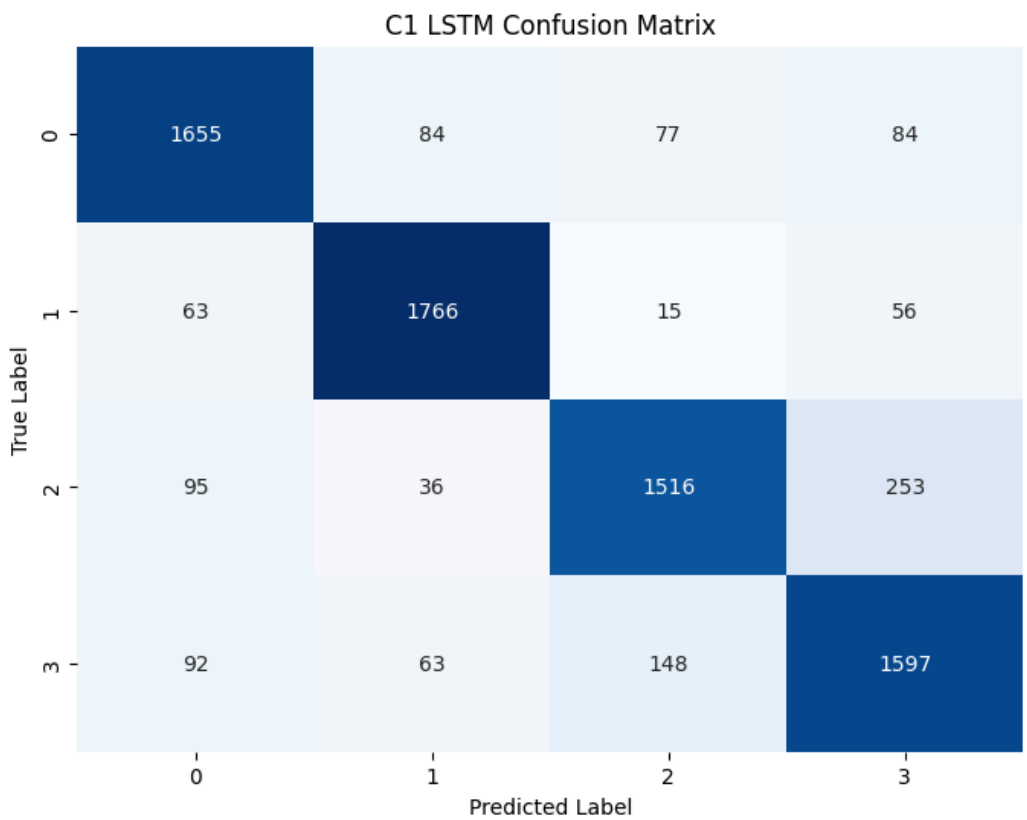
Experiment C(1):

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, None)	0	-
embedding (Embedding)	(None, None, 256)	256,000	input_layer[0][0]
not_equal (NotEqual)	(None, None)	0	input_layer[0][0]
lstm (LSTM)	(None, 32)	36,992	embedding[0][0], not_equal[0][0]
dropout (Dropout)	(None, 32)	0	lstm[0][0]
dense (Dense)	(None, 4)	132	dropout[0][0]

Total params: 293,124 (1.12 MB)
Trainable params: 293,124 (1.12 MB)
Non-trainable params: 0 (0.00 B)



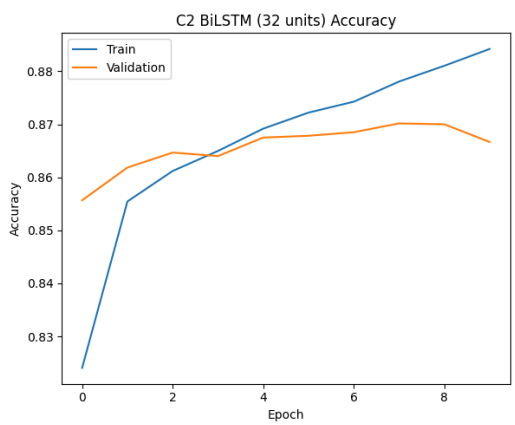


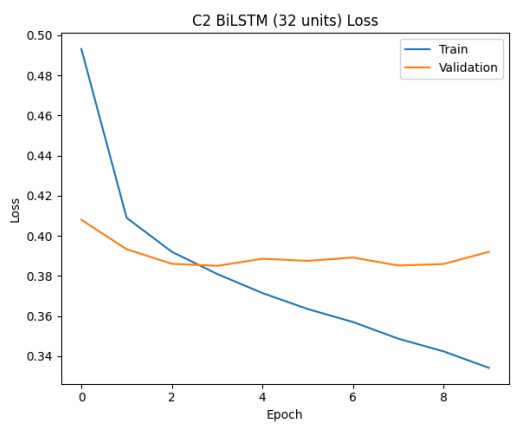
Experiment C(2):

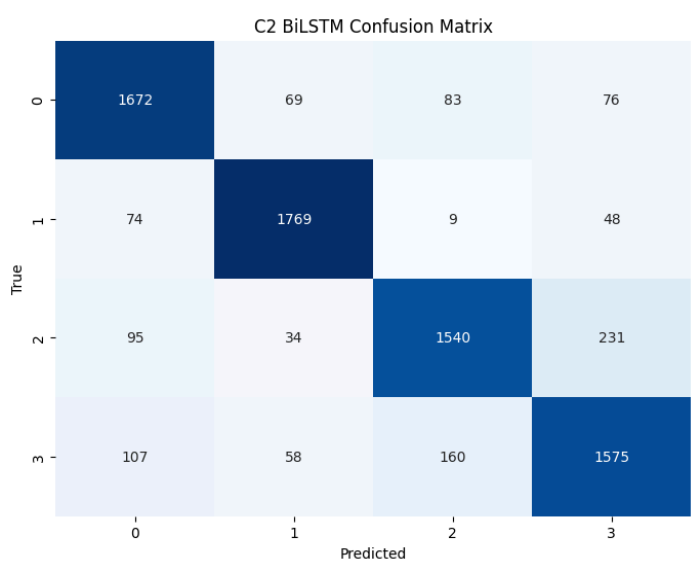
Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, None)	0	–
embedding (Embedding)	(None, None, 256)	256,000	input_layer[0][0]
not_equal (NotEqual)	(None, None)	0	input_layer[0][0]
bidirectional (Bidirectional)	(None, 64)	73,984	embedding[0][0], not_equal[0][0]
dropout (Dropout)	(None, 64)	0	bidirectional[0]…
dense (Dense)	(None, 4)	260	dropout[0][0]

Total params: 330,244 (1.26 MB)
Trainable params: 330,244 (1.26 MB)
Non-trainable params: 0 (0.00 B)





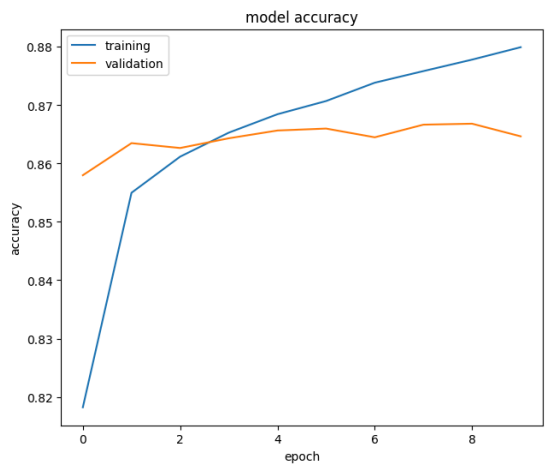


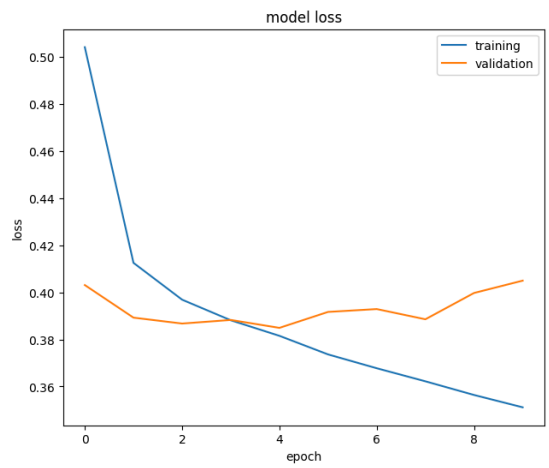
Experiment C(3):

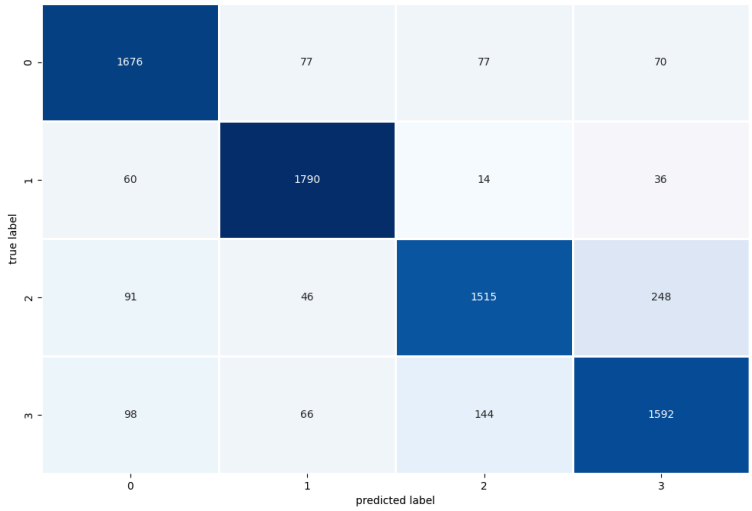
Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, None)	0	–
embedding (Embedding)	(None, None, 256)	256,000	input_layer[0][0]
not_equal (NotEqual)	(None, None)	0	input_layer[0][0]
bidirectional (Bidirectional)	(None, 64)	73,984	embedding[0][0], not_equal[0][0]
dropout (Dropout)	(None, 64)	0	bidirectional[0]…
dense (Dense)	(None, 64)	4,160	dropout[0][0]
dropout_1 (Dropout)	(None, 64)	0	dense[0][0]
dense_1 (Dense)	(None, 4)	260	dropout_1[0][0]

Total params: 334,404 (1.28 MB)
Trainable params: 334,404 (1.28 MB)
Non-trainable params: 0 (0.00 B)





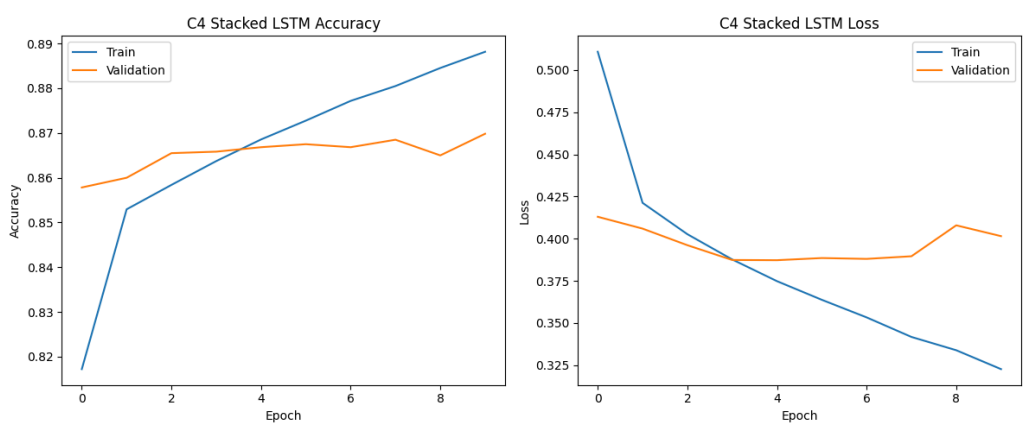


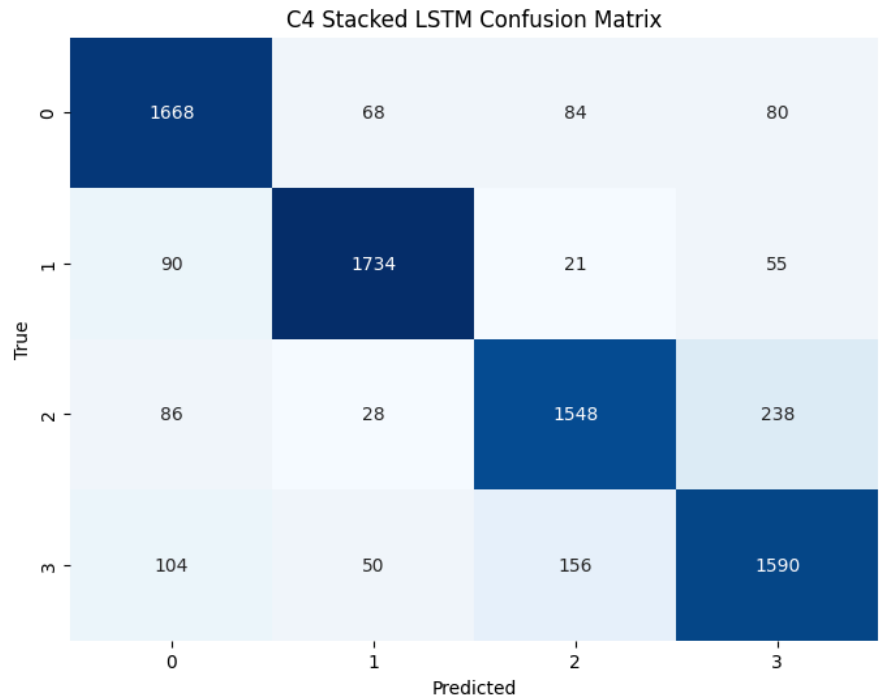
Experiment C(4):

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, None)	0	–
embedding (Embedding)	(None, None, 256)	256,000	input_layer[0][0]
not_equal (NotEqual)	(None, None)	0	input_layer[0][0]
lstm (LSTM)	(None, None, 64)	82,176	embedding[0][0], not_equal[0][0]
dropout (Dropout)	(None, None, 64)	0	lstm[0][0]
lstm_1 (LSTM)	(None, 32)	12,416	dropout[0][0], not_equal[0][0]
dropout_1 (Dropout)	(None, 32)	0	lstm_1[0][0]
dense (Dense)	(None, 4)	132	dropout_1[0][0]

Total params: 350,724 (1.34 MB)
Trainable params: 350,724 (1.34 MB)
Non-trainable params: 0 (0.00 B)



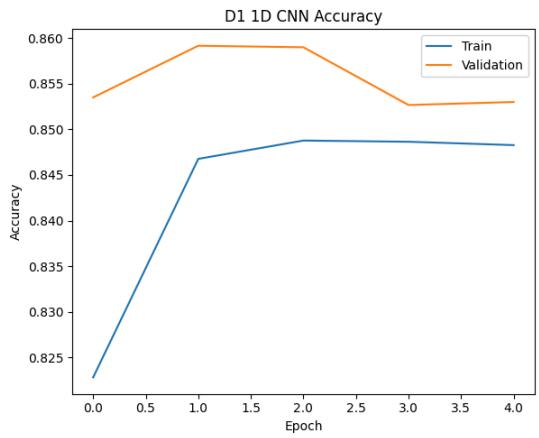


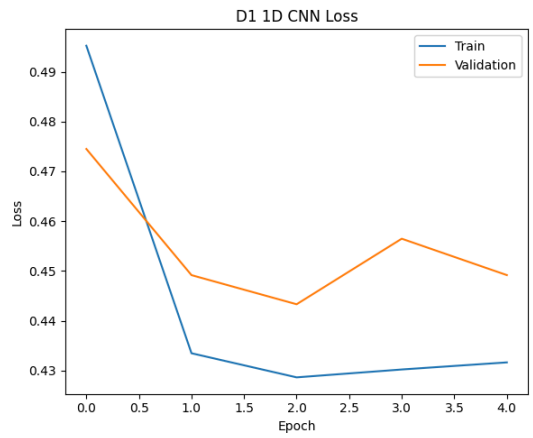
Experiment D(1):

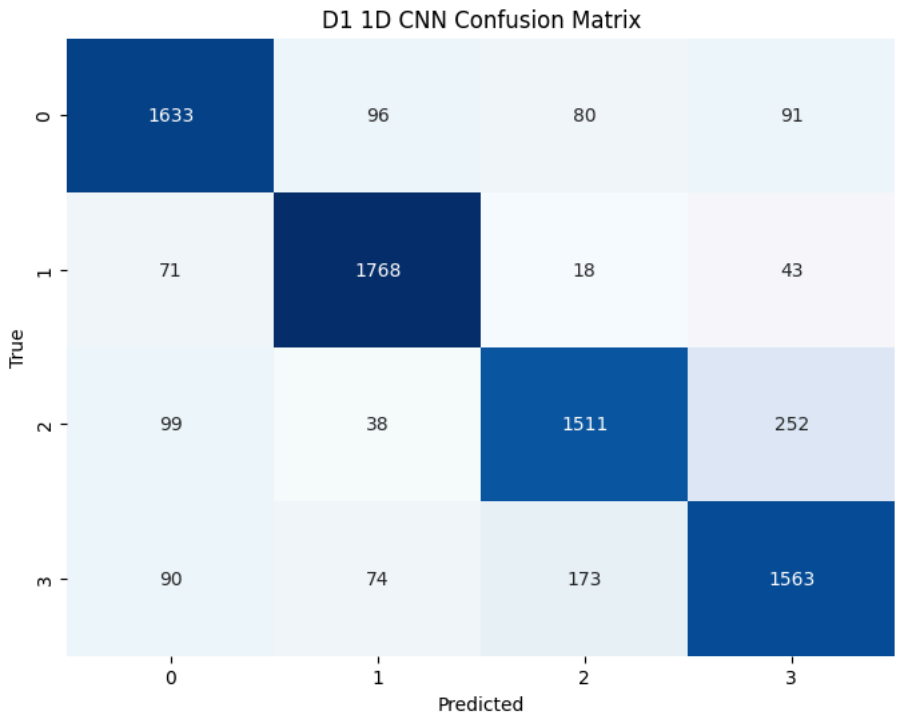
Model: "functional"

Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, None)	0
embedding (Embedding)	(None, None, 256)	256,000
conv1d (Conv1D)	(None, None, 32)	24,608
dropout (Dropout)	(None, None, 32)	0
max_pooling1d (MaxPooling1D)	(None, None, 32)	0
global_max_pooling1d (GlobalMaxPooling1D)	(None, 32)	0
dense (Dense)	(None, 256)	8,448
dense_1 (Dense)	(None, 4)	1,028

Total params: 290,084 (1.11 MB)
Trainable params: 290,084 (1.11 MB)
Non-trainable params: 0 (0.00 B)







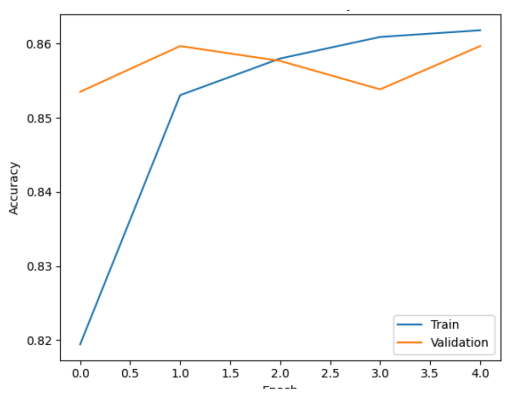
Experiment D2:

Model: "functional"

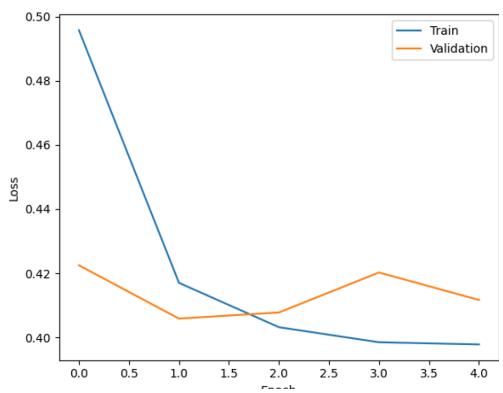
Layer (type)	Output Shape	Param #
input_layer (InputLayer)	(None, None)	0
embedding (Embedding)	(None, None, 256)	256,000
conv1d (Conv1D)	(None, None, 64)	49,216
conv1d_1 (Conv1D)	(None, None, 64)	12,352
dropout (Dropout)	(None, None, 64)	0
max_pooling1d (MaxPooling1D)	(None, None, 64)	0
global_max_pooling1d (GlobalMaxPooling1D)	(None, 64)	0
dense (Dense)	(None, 256)	16,640
dense_1 (Dense)	(None, 4)	1,028

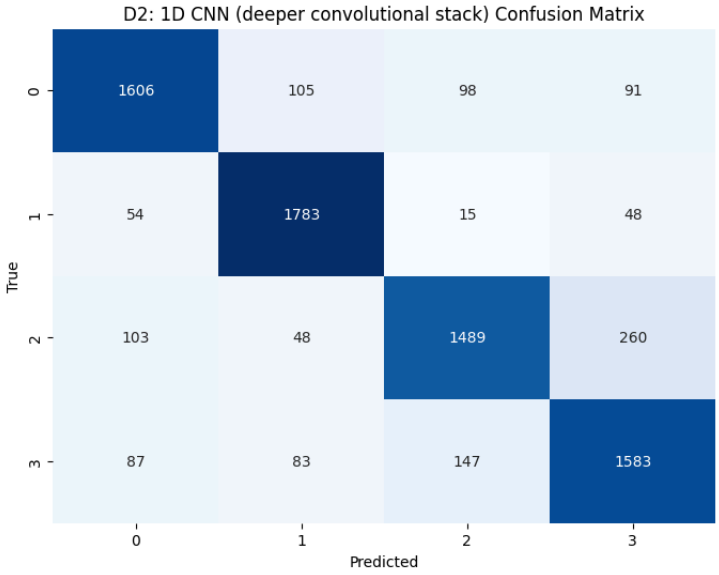
Total params: 335,236 (1.28 MB)
Trainable params: 335,236 (1.28 MB)
Non-trainable params: 0 (0.00 B)

Accuracy



Loss




D2: 1D CNN (deeper convolutional stack) Confusion Matrix
True
Predicted
0
1
2
3
1606
105
98
91
54
1783
15
48
103
48
1489
260
87
83
147
1583